

Система удалённого GUI-тестирования Qt-приложений qtiva-cloud

1. Цель проекта

Создать систему, позволяющую удалённо выполнять автоматические GUI-тесты Qt-приложений без необходимости локальной установки тестовой среды.

Система должна принимать подготовленные артефакты (бинарь приложения и сценарии тестов), запускать тесты в изолированных контейнерах с нужной конфигурацией окружения и возвращать результаты: логи и скриншоты(?).

2. Основные возможности

1. Приём артефактов

- Загрузка архива `app.tar.gz` с бинарём и файлом `manifest.json`.
- Проверка целостности по SHA256.
- Хранение артефактов на сервере до выполнения теста.

2. Удалённый запуск тестов

- Запуск тестов в контейнерах с заданным окружением:
 - ОС: **Ubuntu, Debian, Astra Linux, Manjaro, Arch (?)**.
 - Среда отображения: **X11** или **Wayland**.
 - Версия Qt: **5.15**.
- Возможность параллельного выполнения нескольких прогонов на одном хосте.
- Изоляция каждого прогона в отдельном контейнере.
- Поддержка установки дополнительных зависимостей, указанных в `manifest.json`.

3. Выбор окружения

- Пользователь может выбрать тип ОС и графической подсистемы (`x11` / `wayland`).
- Внутри каждой ОС поддерживаются предсобранные образы окружений (`qtiva-runner:os-qt5.15-x11` , `qtiva-runner:os-qt5.15-wayland`).

4. Отчёты

- Система формирует папку результатов (`/artifacts`) с:
 - логами выполнения (`run.log`);
 - скриншотами (`.png`) (?);

- кодом возврата (`passed` , `failed` , `error` , `timeout`).
- Доступ к результатам через HTTP API.

3. Архитектура системы

Компонент	Описание
Manager (API-сервис)	Принимает запросы от пользователей, управляет артефактами и прогонами.
Agent	Исполнитель, который управляет запуском контейнеров на локальном хосте. Может выполнять несколько задач параллельно.
Runner containers	Временные контейнеры, в которых поднимается виртуальный X-сервер (или Wayland), запускается Qt-приложение и выполняется тест.
Хранилище данных	Локальная файловая система (<code>./data/</code>) для артефактов и результатов. В будущем — S3/MinIO.

4. Функциональные модули

4.1 API Manager

Метод	Путь	Назначение
POST	<code>/artifacts</code>	Загрузка архива с приложением и манифестом
POST	<code>/runs</code>	Запуск теста по <code>artifact_id</code> с параметрами окружения
GET	<code>/runs/{id}</code>	Получение статуса выполнения и ссылок на артефакты

Статусы прогона: `pending` → `running` → (`passed` | `failed` | `error` | `timeout`)

Авторизация: простой Bearer-токен (`Authorization: Bearer <token>`).

4.2 Agent

- Поддерживает **N** параллельных воркеров (по количеству CPU или конфигурации).
- Каждый воркер:
 1. Берёт задачу из локальной очереди.
 2. Запускает контейнер с нужным образом (`qtiva-runner:<os>-qt<ver>-<display>`).
 3. Монтирует:

- `/input` → архив артефакта;
 - `/artifacts` → каталог для логов и скриншотов.
4. Выполняет команду из `manifest.json`.
 5. Стримит логи в файл `run.log`.
 6. По завершении выгружает артефакты и обновляет статус.
- Контейнеры создаются и удаляются автоматически после завершения.

5. Формат `manifest.json`

```
{
  "run_cmd": "./myapp --test-suite smoke",
  "timeout_sec": 600,
  "needs_display": true,
  "qt_version": "5.15",
  "os": "ubuntu",
  "display": "x11",
  "packages": {
    "apt": ["libpulse0", "libxcb-icccm4"]
  },
  "env": {
    "LC_ALL": "C",
    "QT_QPA_PLATFORM": "xcb"
  }
}
```

6. Образы окружений

Каждый базовый образ `qtiva-runner:<os>-qt<ver>-<display>` содержит:

- Соответствующую ОС (Ubuntu, Debian, Astra, Arch, Manjaro);
- Qt-runtime нужной версии;
- Xvfb и/или Wayland-среду;
- Шрифты и базовые пакеты;
- Утилиту `qtiva` внутри (`/qtiva/runner`).

Перед запуском теста агент может доустановить зависимости:

```
apt-get update && apt-get install -y <список из manifest.json>
```

или аналогично через `pacman`, `dnf`, `apt`, в зависимости от ОС образа.

7. Безопасность и изоляция

- Каждый прогон выполняется в отдельном контейнере.
- Контейнер запускается с:
 - `readOnlyRootFilesystem: true`
 - `--cpus` и `--memory` лимитами
 - `NetworkMode: none` (если не требуется сеть)
- После завершения:
 - контейнер удаляется;
 - временные файлы очищаются;
 - артефакты переносятся в `./data/runs/<run_id>/`.

8. Параллельность и ресурсы

- Один агент-процесс обслуживает несколько задач одновременно.
- Количество одновременно работающих контейнеров (`concurrency`) задаётся в конфиге (`qtiva-agent.yml`):

```
concurrency: 4
default_cpu_limit: 2
default_memory_limit: 2048
```

- Планировщик агента сам регулирует загрузку по семафору.

9. Технологический стек

Компонент	Технология
Язык	Go (возможен Python/FastAPI как альтернатива)
Контейнеризация	Docker-outside-of-Docker (монтирование <code>/var/run/docker.sock</code>)
Хранилище данных	Локальная файловая система (<code>./data</code>)
Логи и артефакты	Текстовые файлы и изображения
CI-интеграция	HTTP API (<code>curl</code> / GitHub Actions / GitLab CI)

10. Пример потока работы

1. Пользователь собирает приложение и тесты.

2. Создаёт архив:

```
app.tar.gz → { myapp, manifest.json, tests/ }
```

3. Отправляет:

```
curl -F "file=@app.tar.gz" http://qtiva:8080/artifacts
```

4. Запускает тест:

```
curl -H "Content-Type: application/json" \  
-d '{"artifact_id":"art_001","os":"ubuntu","display":"x11"}' \  
http://qtiva:8080/runs
```

5. Проверяет статус:

```
curl http://qtiva:8080/runs/run_001
```

6. После завершения получает ссылки на `run.log` и `screenshots/*.png`.